MACHINE LEARNING

LAB RECORD

ITA0614 | EXPERIMENT 5

EXPERIMENT STATEMENT

Write a program for Implementation of K-Nearest Neighbours (K-NN) in Python.

STUDENT NAME	REGISTER NUMBER
P H Devesh	**1911260236**
SUBJECT CODE	**SUBJECT NAME**
ITA0614	**Machine Learning**

Five program demonstrations • Executed Python examples • Verified output

SUB-PROGRAM 1

K-NN Classification on a Small Numeric Dataset

Problem Statement / Program Title

K-NN Classification on a Small Numeric Dataset.

Aim

Implement K-Nearest Neighbours from scratch using Euclidean distance to classify a new numeric sample.

Algorithm

1. Store the labelled training points and choose the value of K.
2. Calculate Euclidean distance from the query point to every training point.
3. Sort training points by increasing distance and select the K nearest neighbours.
4. Count the labels among the selected neighbours.
5. Return the most frequent label and show the nearest neighbours and predicted class.

Program

```
from math import sqrt
from collections import Counter

training_data = [
    ([1.0, 1.0], "A"),
    ([1.5, 2.0], "A"),
    ([3.0, 4.0], "B"),
    ([3.5, 4.5], "B"),
    ([1.2, 0.8], "A"),
]
query = [2.8, 3.8]
k = 3

def euclidean(a, b):
    if len(a) != len(b):
        raise ValueError("Points must have the same number of features")
    return sqrt(sum((x-y)**2 for x, y in zip(a, b)))

distances = [(euclidean(query, point), label, point)
             for point, label in training_data]
nearest = sorted(distances, key=lambda item: item[0])[:k]
prediction = Counter(label for _, label, _ in nearest).most_common(1)[0][0]

print("Training points:", len(training_data))
print("Query point:", query)
print(f"K = {k}")
print("Nearest neighbours:")
for distance, label, point in nearest:
    print(f"  {point} -> {label}, distance={distance:.3f}")
print("Predicted class:", prediction)
```

Input / Dataset

Five labelled two-feature points (class A or B); query point [2.8, 3.8] and K=3. All data are embedded in the program.

Output

```
Training points: 5
Query point: [2.8, 3.8]
K = 3
Nearest neighbours:
  [3.0, 4.0] -> B, distance=0.283
  [3.5, 4.5] -> B, distance=0.990
  [1.5, 2.0] -> A, distance=2.220
Predicted class: B
```

Result

The from-scratch K-NN implementation selected the three nearest points and predicted class B by majority vote.

SUB-PROGRAM 2

K-NN on the Iris Dataset

Problem Statement / Program Title

K-NN on the Iris Dataset.

Aim

Classify Iris flowers with K-NN and evaluate performance on a held-out test set.

Algorithm

1. Load the Iris dataset containing four measurements and three species classes.
2. Split the data into training and test subsets using stratified sampling.
3. Standardize feature values using the training data only.
4. Train a K-NN classifier with K=5.
5. Predict labels for the test samples and calculate accuracy and a confusion matrix.
6. Classify one sample and display the predicted species.

Program

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25,
    random_state=42, stratify=iris.target
)
model = make_pipeline(
    StandardScaler(),
    KNeighborsClassifier(n_neighbors=5)
)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
sample = [[5.1, 3.5, 1.4, 0.2]]

print("Dataset: Iris")
print("Training samples:", len(X_train))
print("Test samples:", len(X_test))
print("K:", 5)
print("Test accuracy:", f"{accuracy_score(y_test, predictions)*100:.2f}%")
print("Confusion matrix (rows=true, columns=predicted):")
print(confusion_matrix(y_test, predictions))
print("Sample measurements:", sample[0])
print("Predicted species:", iris.target_names[model.predict(sample)[0]])
```

Input / Dataset

The Iris dataset has 150 flower samples, four measurements, and three species. A stratified 75/25 split yields 112 training and 38 test records.

Output

```
Dataset: Iris
Training samples: 112
Test samples: 38
K: 5
Test accuracy: 92.11%
Confusion matrix (rows=true, columns=predicted):
[[12  0  0]
 [ 0 13  0]
 [ 0  3 10]]
Sample measurements: [5.1, 3.5, 1.4, 0.2]
Predicted species: setosa
```

Result

The K-NN model reached 92.11% accuracy on the held-out Iris test set and predicted the sample measurements as setosa.

SUB-PROGRAM 3

K-NN for Binary Classification

Problem Statement / Program Title

K-NN for Binary Classification.

Aim

Use K-NN for a reproducible binary classification problem and report test accuracy and class predictions.

Algorithm

1. Generate a synthetic binary dataset with numerical features.
2. Split the dataset into training and test sets with stratification.
3. Standardize features inside a pipeline to prevent test-set leakage.
4. Fit a K-NN classifier with K=7.
5. Predict test labels and calculate accuracy.
6. Print actual and predicted labels for a few test records.

Program

```
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

X, y = make_classification(
    n_samples=200, n_features=4, n_informative=3,
    n_redundant=0, class_sep=1.2, random_state=10
)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=10, stratify=y
)
model = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7))
model.fit(X_train, y_train)
pred = model.predict(X_test)

print("Dataset: synthetic binary classification")
print("Training samples:", len(X_train))
print("Test samples:", len(X_test))
print("K:", 7)
print("Test accuracy:", f"{accuracy_score(y_test, pred)*100:.2f}%")
print("First 10 actual labels:   ", y_test[:10].tolist())
print("First 10 predicted labels:", pred[:10].tolist())
```

Input / Dataset

Synthetic illustrative dataset of 200 records with four numeric features and two classes. A stratified 75/25 split yields 150 training and 50 test records.

Output

```
Dataset: synthetic binary classification
Training samples: 150
Test samples: 50
K: 7
Test accuracy: 90.00%
First 10 actual labels:    [1, 0, 1, 0, 0, 1, 0, 0, 1, 1]
First 10 predicted labels: [1, 0, 1, 1, 0, 1, 0, 0, 1, 1]
```

Result

The K-NN model achieved 90.00% accuracy on the held-out synthetic test set. The score is specific to the generated data and fixed split.

SUB-PROGRAM 4

K-NN Regression for Continuous Value Prediction

Problem Statement / Program Title

K-NN Regression for Continuous Value Prediction.

Aim

Demonstrate K-nearest-neighbour regression by predicting a continuous target from nearby training points.

Algorithm

1. Define one-dimensional training features and continuous target values.
2. Fit a KNeighborsRegressor with K=3.
3. Use the three nearest training points to estimate the query target by averaging their target values.
4. Predict target values for the query points.
5. Display the query features and predicted continuous values.

Program

```
import numpy as np
from sklearn.neighbors import KNeighborsRegressor

X_train = np.array([[1.0], [2.0], [3.0], [4.0], [5.0], [6.0]])
y_train = np.array([10.0, 20.0, 29.0, 41.0, 50.0, 61.0])
X_query = np.array([[2.5], [4.5]])
model = KNeighborsRegressor(n_neighbors=3, weights="uniform")
model.fit(X_train, y_train)
predictions = model.predict(X_query)

print("Training feature values:", X_train.ravel().tolist())
print("Training targets:", y_train.tolist())
print("K:", 3)
for x, yhat in zip(X_query.ravel(), predictions):
    print(f"Predicted target for x={x:.1f}: {yhat:.2f}")
```

Input / Dataset

Six one-dimensional training inputs with continuous target values; query inputs are 2.5 and 4.5. Uniform averaging is used over K=3 neighbours.

Output

```
Training feature values: [1.0, 2.0, 3.0, 4.0, 5.0, 6.0]
Training targets: [10.0, 20.0, 29.0, 41.0, 50.0, 61.0]
K: 3
Predicted target for x=2.5: 19.67
Predicted target for x=4.5: 40.00
```

Result

K-NN regression predicted continuous target values by averaging the target values of the three nearest training points.

SUB-PROGRAM 5

Compare K Values on the Digits Dataset

Problem Statement / Program Title

Compare K Values on the Digits Dataset.

Aim

Compare test accuracy for several K values in K-NN classification using a fixed train-test split.

Algorithm

1. Load the handwritten digits dataset and scale pixel features to the range 0 to 1.
2. Split data into training and test subsets with stratification.
3. Train K-NN classifiers for K values 1, 3, 5, and 7.

4. Predict the held-out test labels for each K.

5. Calculate and print test accuracy for every K.

6. Identify the K with the highest measured accuracy on this split.

Program

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

digits = load_digits()
X = digits.data / 16.0
y = digits.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)
scores = {}
for k in [1, 3, 5, 7]:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    scores[k] = accuracy_score(y_test, predictions)

print("Dataset: handwritten digits")
print("Training samples:", len(X_train))
print("Test samples:", len(X_test))
for k, score in scores.items():
    print(f"K={k}: test accuracy={score*100:.2f}%")
best_k = max(scores, key=scores.get)
print("Best K for this split:", best_k)
```

Input / Dataset

The built-in digits dataset has 1,797 handwritten digit samples with 64 pixel features. A fixed stratified 75/25 split yields 1,347 training and 450 test samples.

Output

```
Dataset: handwritten digits
Training samples: 1347
Test samples: 450
K=1: test accuracy=98.44%
K=3: test accuracy=98.44%
K=5: test accuracy=98.44%
K=7: test accuracy=98.00%
Best K for this split: 1
```

Result

K values 1, 3, and 5 tied at 98.44% test accuracy on this digits split; K=1 is reported as the first best value. The comparison is specific to this split.